Static Hand-Sign Alphabet Classification using Sign Language MNIST

Kunruethai Patimapornchai
Faculty of ICT, Mahidol University,
Nakhon Pathom, Thailand 73170
kunruethai.pat@student.mahidol.ac.th

Thanakit Rattikanchalakorn
Faculty of ICT, Mahidol University,
Nakhon Pathom, Thailand 73170
thanakit.rat@student.mahidol.ac.th

Teemtat Weerarattanamanee
Faculty of ICT, Mahidol University,
Nakhon Pathom, Thailand 73170
teemtat.wee@student.mahidol.ac.th

Abstract *– This project evaluates multiple Convolutional Neural Network (CNN) architectures for real-time sign language alphabet recognition. Following an iterative methodology, three distinct models were developed and compared via grid search: a custom Baseline CNN, a DenseNet architecture, and a pretrained MobileNetV2. The Baseline CNN demonstrated superior performance, achieving over 99% validation accuracy, while complex transfer learning models struggled due to domain gaps. The final model was successfully deployed using an OpenCV and MediaPipe pipeline for live inference.*

I. INTRODUCTION

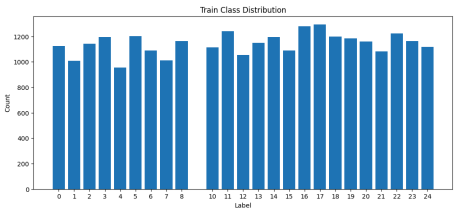
Communication between the Deaf and Hard-of-Hearing (DHH) community and the hearing population often faces challenges. While American Sign Language (ASL) is a primary language for millions, it remains inaccessible, especially for people who haven't studied it. The project addresses the challenge of static ASL alphabet recognition, translating hand signs into alphabets to facilitate basic communication.

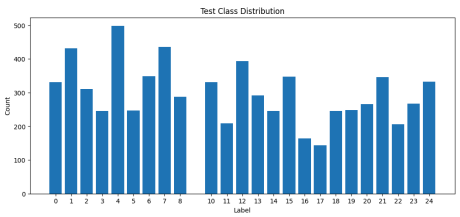
The project aims to enhance digital inclusiveness by developing a low-cost sign language recognition system using a standard webcam. However, implementing the project in real-time presents significant challenges, including environmental noise, the high visual similarity between letters, and the hand signs with motion in some alphabets. To address these difficulties, this work contributes a CNN models optimization and a comparative analysis of CNN models to achieve a classification model with the best performance.

II. METHODOLOGY

A. Dataset: Sign Language MNIST

1. **Dataset Description:** The Sign Language MNIST dataset is publicly available dataset hosted on Kaggle.
    a. **Features:** Each sample consists of a 28×28 grayscale image.
    b. **Format:** CSV format, where each row contains a label and 784 pixel (28×28 grid) intensity values.
    c. **Training Set:** 27,455 samples.
    d. **Test Set:** 7,172 samples.
    e. **Classes:** 24 classes corresponding to the English alphabets (Excluding J and Z).





Note: *The letters J and Z require motion to be understood. Since this is a static image dataset, these two letters are excluded.*

2. **Preprocessing Steps**
    a. **Normalization:** Pixel values are converted to `float32` and divided by 255.0 to scale them in [0,1] range, ensuring numerical stability.
    b. **Reshaping:** The 1D arrays of 784 pixels are reshaped into 4D tensors with dimensions (N,28,28,1), where N is the number of samples and 1 represents the grayscale channel.
    c. **Label Re-indexing:** Since labels 9 (J) and 25 (Z) are missing, the original label set is non-contiguous. We mapped the raw labels to a 0-based continuous index (0–23) using a custom `LABEL_MAP`.
    d. **Data augmentation:** In order to improve model generalization against variations in hand positioning, Data augmentation pipeline was integrated using `RandomRotation` (0.08), `RandomTranslation` (0.08), and `RandomZoom` (0.10), simulating different hand positionings.
3. **Data Issues**
    a. **Resolution Constraints:** The 28×28 resolution is significantly lower than a standard webcam feed. Therefore, a precise "Region of Interest" (ROI) crop during live deployment is required to ensure the hand occupies the majority of the frame.
    b. **Domain Gap:** The dataset images are grayscale and tightly cropped with specific lighting. Real-world webcam captures color images with diverse lighting conditions, which can reduce the performance if not handled by additional preprocessing (like grayscaling and resizing).
    c. **Missing Gestures:** The exclusion of 'J' and 'Z' notably impacts some real world scenarios, making the model incomplete.

B. Model Development

1. **Iteration 1: Baseline CNN.** The initial model was constructed using three sequential convolutional blocks featuring Batch Normalization, Max Pooling, and Dropout (0.25), followed by a dense layer with high dropout (0.50). This model was designed to be a performance baseline.
2. **Iteration 2: DenseNet (Basic).** To address potential overfitting in the Baseline and explore parameter efficiency, a custom DenseNet was implemented. The goal was to prevent the loss of information in deeper layers of the network. By linking each layer to all the layers before it, the model can effectively reuse important features.
3. **Iteration 3: MobileNetV2 (Transfer Learning).** To test if pre-trained ImageNet weights could accelerate convergence or improve real-world camera robustness, MobileNetV2 was utilized. The images were upscaled to 96x96 and channel-duplicated to RGB to satisfy network input requirements.

III. RESULTS

A. Experiment Setup

All models were compiled using the Adam optimizer and Sparse Categorical Crossentropy loss. The primary evaluation metrics were accuracy and validation loss. A grid search was conducted to optimize hyperparameters, such as learning rate and drop out rate. Training utilized Early Stopping (patience=3 to 5) to mitigate overfitting.

B. Performance Metrics

Model performance was evaluated using validation accuracy and crossentropy loss. The optimal hyperparameter configurations for each architecture are detailed in the grid search results below.

1. **Baseline CNN:** The model demonstrated extreme stability across all configurations, consistently exceeding 99.7% validation accuracy. The optimal setup (LR=0.0005, Dropout=0.30, Filters=[64, 128, 256]) peaked at 99.96% validation accuracy and minimal loss (0.0057) by epoch 8.

Learning rate	Dropout rate	Filters	Best Accuracy	Best Loss	Epochs
0.0005	0.3	(64, 128, 256)	99.96	0.0057	8
0.0005	0.25	(64, 128, 256)	99.93	0.0081	7
0.0005	0.4	(32, 64, 256)	99.93	0.0065	13
0.0005	0.25	(32, 64, 256)	99.85	0.0056	9
0.0005	0.3	(32, 64, 256)	99.71	0.0102	12
0.0005	0.4	(64, 128, 256)	99.49	0.0523	7

2. **DenseNet:** The search revealed high sensitivity to regularization. While a dropout of 0.30 allowed the model to reach a 90.93% validation accuracy (with LR=0.0010), simply increasing the dropout to 0.40 caused performance to drop below 52% across all tested learning rates.

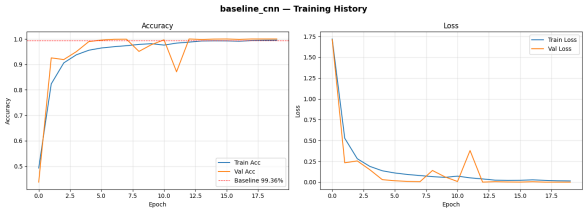
Learning rate	Growth rate	Dropout rate	Best Accuracy	Best Loss	Epochs
0.001	12	0.3	90.93	0.2464	12
0.0005	12	0.3	84.23	0.421	14
0.001	12	0.4	51.38	1.5462	8
0.0005	12	0.4	40.35	1.9205	7

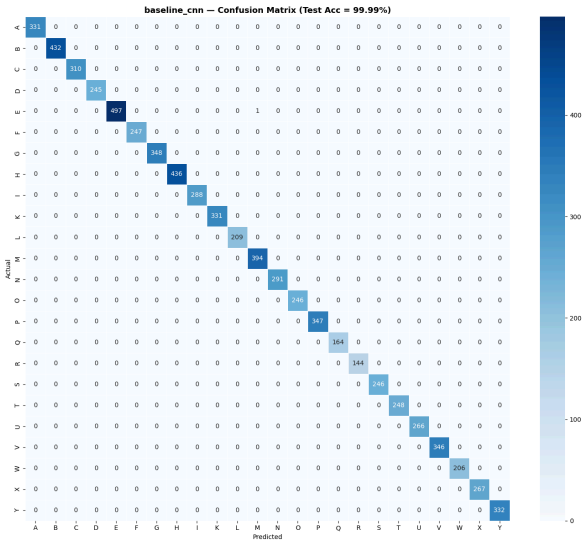
3. **MobileNetV2:** Hyperparameter tuning showed no improvements. Across all combinations of learning rates (0.0010, 0.0001), dropout rates, and unfreezing up to 20 backbone layers, the validation accuracy remained at 4.70%.

Learning rate	Dropout rate	Unfreeze Layers (from last)	Best Accuracy	Best Loss	Epochs
0.001	0.2	0	4.7	3.1755	5
0.001	0.3	0	4.7	3.1755	4
0.001	0.3	20	4.7	3.1753	4
0.0001	0.2	0	4.7	3.1766	8
0.0001	0.2	20	4.7	3.4483	5
0.0001	0.3	0	4.7	3.1772	4
0.0001	0.3	20	4.66	3.9298	5
0.001	0.2	20	4.37	17.2634	5

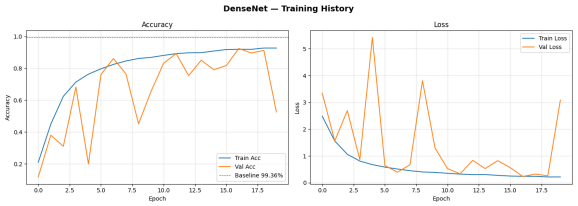
C. Model Comparison

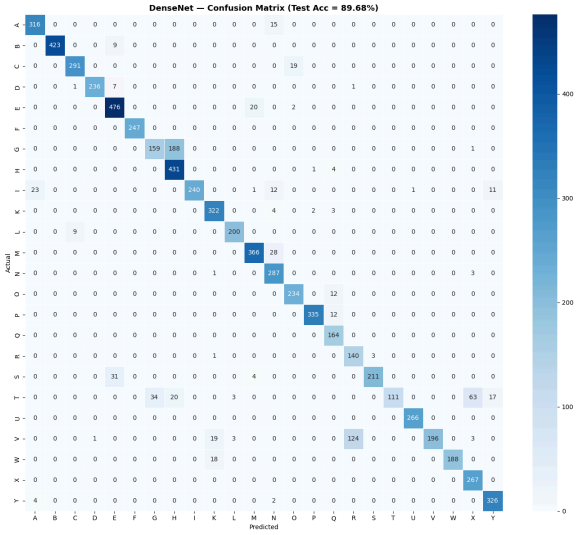
1. **Baseline CNN:** The baseline model significantly outperformed the others, achieving a test accuracy of 99.36% and a loss of 0.0472. The learning curves illustrated convergence with minimal generalization gap between train and validation accuracy, indicating that the dropout layers successfully prevented overfitting.



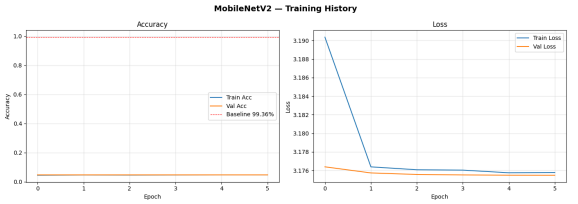


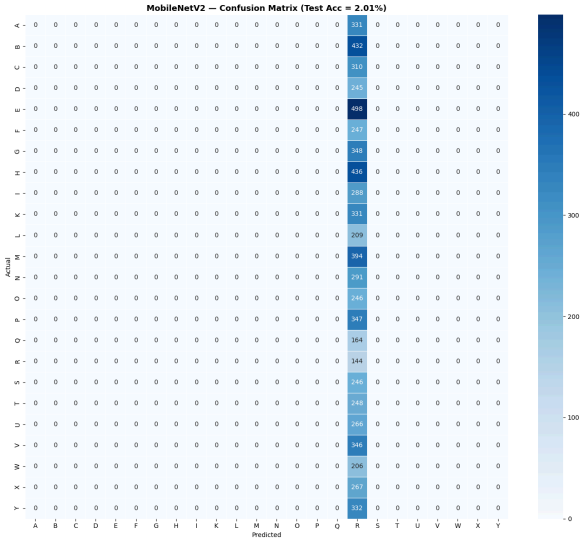
2. **DenseNet:** The optimal DenseNet configuration (growth rate=12, dropout=0.30) achieved lower overall performance than the baseline. While the dense connections ensured strong gradient flow, the added model complexity was unnecessary for the simple 28x28 grayscale feature space.





3. **MobileNetV2:** This model demonstrated severe underfitting. Despite fine-tuning the unfreezing layers and adjusting the learning rate, the network failed to adapt. Because MobileNetV2 was pre-trained on complex, high-resolution color images, its learned features are incompatible with the simple, grayscale structure of our sign language dataset.





IV. DEPLOYMENT & LIVE DEMO

The final model was set up in a live video program using OpenCV and MediaPipe. First, the system finds the hand and cuts out a square box around it, adding a 30-pixel border to make sure the whole hand fits. To help the model see clearly, the image is cleaned up before making a prediction. Then, the system looks at a history of the last 5 frames to output a steady final answer.

However, even with these helpful steps, live webcams can introduce challenges. The system can still make false predictions depending on lighting and background.

Link to source code:

https://github.com/teemtat/sign-language-DeepLearning.git

V. CONCLUSIONS

The primary objective of this project was to develop and deploy an evidence-based deep learning solution for real-time sign language recognition. In pursuit of this goal, the experimental evidence confirms that custom, lightweight CNNs are far more effective for the Sign Language MNIST dataset than complex, pre-trained architectures. The Baseline CNN provided the best trade-off between computational cost and predictive accuracy. On the other hand, the failure of MobileNetV2 highlights the critical importance of matching model capacity and pre-training domains to the specific characteristics of the target dataset. Ultimately, with the success of Baseline CNN, the project fulfilled its core objective of delivering a practical machine learning solution.